Projet MLOps

Modalité du projet

Groupe : 4 à 5 personnes
Evaluation : POC avec **présentation de groupe** du projet en fin de journée

Contexte

L'objectif de ce projet est de mettre en application des principes de MLOps sur un use case précis utilisant du machine learning supervisé ou non supervisé.

Cela inclut donc une des catégories suivantes :

- classification binaire ou multi-class (multi label ou non)
- régression simple ou multivariée
- clustering
- détection d'anomalie

Le(s) type(s) de données que vous traiterez seront selon votre use case :

- tabulaires
- spatio-temporelles
- texte
- graphe
- séries temporelles
- image
- vidéo
- signal
- etc.

Idéalement, une source de données en live / streaming serait parfaite pour ce type de projet. Mais elle nécessite souvent des accès payants ou plus complexes (APIs, Flux RSS, etc.).

Le choix de votre use case est **libre** : vous pouvez développer une idée d'un projet de votre choix, ou bien suivre un de ces exemples :

- **Classification de fraude bancaire** : prédire si une transaction bancaire est frauduleuse ou non.
  Données : https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
- **Reconnaissance des émotions sur des images** : identifier l'émotion dans les images de visages.
  Données : https://www.kaggle.com/datasets/msambare/fer2013
- **Catégorisation de textes** : classifier les articles de presse en 20 catégories différentes.
  Données : https://www.kaggle.com/datasets/au1206/20-newsgroup-original ou des flux RSS en live
- **Localisation indoor** : localiser à partir de WLAN/WiFi fingerprinting
  https://archive.ics.uci.edu/dataset/310/ujiindoorloc
- **Prédiction du prix des maisons** : prédire le prix des maisons en fonction de diverses caractéristiques.
  Données : https://www.kaggle.com/datasets/harlfoxem/housesalesprediction
- **Estimation de la consommation d'énergie** : prédire la consommation d'énergie basée sur des mesures horaires.
  Données :
  https://archive.ics.uci.edu/ml/datasets/Individual+household+electric+power+consumption
- **Segmentation de clients** : segmenter les clients d'un commerce de gros en fonction de leurs habitudes d'achat.
  Données : https://archive.ics.uci.edu/ml/datasets/Wholesale+customers
- **Clustering des documents textuels** : clustering de nouvelles en fonction de leur contenu pour regrouper des articles similaires.
  Données : https://www.kaggle.com/datasets/uciml/news-aggregator-dataset
- **Détection d'anomalies dans des transactions financières** : identifier des transactions financières frauduleuses.
  Données : https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
- **Détection d'anomalies dans les données de température d'un système industriel** : Identifier les anomalies dans les systèmes hydrauliques à partir de leurs capteurs.
  Données :
  https://archive.ics.uci.edu/ml/datasets/Condition+monitoring+of+hydraulic+systems
- etc.

Pour la gestion des données, vous pouvez par exemple faire un DAG d'extraction de données (ex: avec Airflow) qui va lire ces données statiques ou récupérer des données en live (API, flux RSS, etc.) et les sauvegarder dans un bucket S3 ou BDD. A vous de voir les pipelines les plus adaptés à la situation.

Le choix des métriques de performance se fera en fonction de votre use case.

Ressources disponibles

Nous distinguerons pour l'exercice seulement **deux environments** :

- votre environnement de "**dev**" en local pour vos tests / développements. Faites en sorte que tous les services nécessaires (Airflow, MLflow, etc.) tournent en local pour les phases de dev. N'hésitez pas à passer par un Docker Compose pour initialiser et lancer proprement votre environnement de dev.

- un environnement de "**production**", oú l'entièreté de vos services nécessaires pour votre projet (Airflow,, etc.) doit tourner sur votre cluster kubernetes en local (cf : Docker Desktop ou Minikube) ou sur le Cloud si vous avez des crédits disponibles sur AWS, GCP ou Azure. A noter que faire tourner votre projet sur le Cloud est un "nice to have" et n'est donc pas obligatoire. En ce qui concerne la notation, vous ne perdrez pas de points si vous ne pouvez que rester sur votre cluster Kubernetes en local.
    - Vous pourrez si vous le souhaitez, utiliser le MLflow online

Pensez à distinguer les exécutions sur les environnements (dev ou production) via la **CI/CD**. N'hésitez pas à utiliser des **Helm Chart** existants pour déployer vos services sur votre cluster Kubernetes.

Objectifs

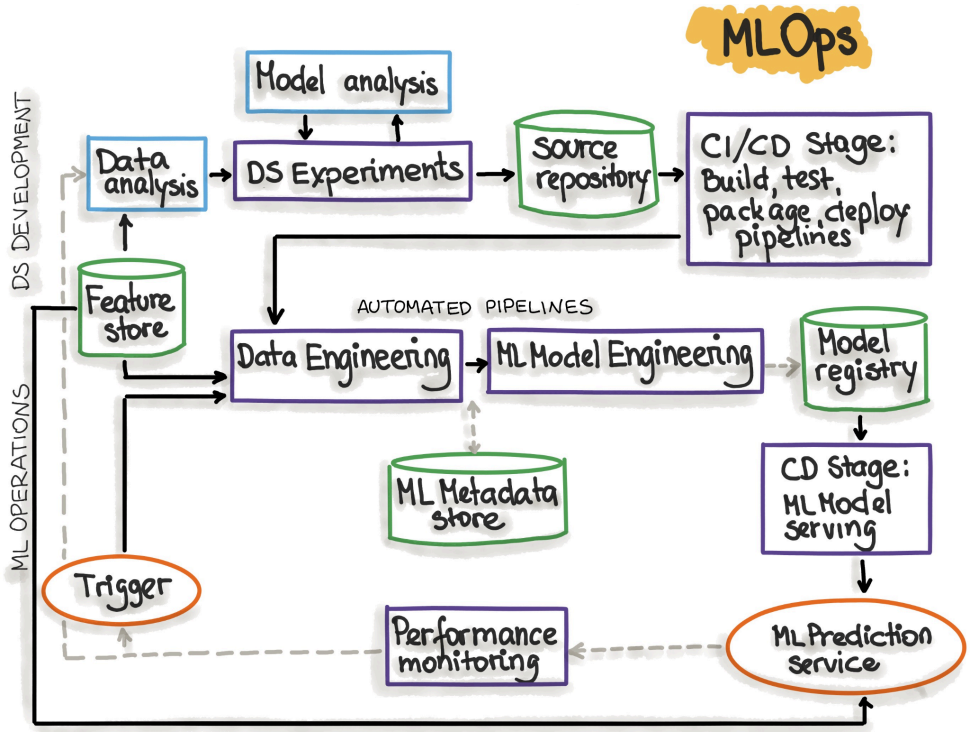
1. **Extraire et prétraiter les données** : télécharger les images depuis les URLs, les nettoyer et les préparer pour l'entraînement.
2. **Construire un modèle de classification** : développer un modèle de ML/DL. Utilisez la techno la plus adaptée à vos besoins : Scikit-learn, AutoGluon, FastAI, PyTorch, Tensorflow, etc.

   Rester **simple**, pas besoin d'un modèle ultra performant pour ce projet

3. **Stocker le modèle sur un model registry** : une fois entraîné, le modèle devra être sauvegardé dans un model registry (ex : un bucket **S3**).
4. **Créer un pipeline de réentraînement avec Apache Airflow** : si besoin et si utile, mettre en place un ou des pipelines qui récupèrent les nouvelles données, met à jour le modèle et le redéploie, etc.
5. **Suivre les modèles et les expériences avec MLFlow** : utiliser MLFlow pour enregistrer les performances et suivre les versions du modèle.
6. **Développer une API** : construire une API permettant d'envoyer une image et de recevoir une prédiction. Utilisez la techno la plus adaptée à vos besoins : FastAPI, Flask, KServe, Torch Serve, etc.

7. **Créer une WebApp pour interagir avec le modèle** : développer une interface utilisateur simple pour visualiser les résultats des prédictions.
   Utilisez la techno la plus adaptée à vos besoins : Gradio, Streamlit, Voila, etc.
8. Gérer l'**Entraînement Continue (CT)** : ajouter un ou plusieurs DAG Airflow avec des triggers que vous définirez (nouvelle données, entraînement hebdomadaire, performances du modèle en baisse, etc.) pour réentraîner et déployer automatiquement un nouveau modèle. Ajouter des checks / tests pour vérifier que le nouveau modèle est mieux que l'ancien.
9. **Dockeriser** votre API et **déployer** là sur un cluster Kubernetes en local (Docker Desktop ou MiniKube). Utilisez la **CI/CD** de **GitHub Actions** pour celà.
10. **Versionner et documenter le projet sur GitHub** : tout le projet devra être hébergé sur **GitHub**, avec une structure de fichiers propre et une documentation claire.

11. Gérer des **données en live** (flux RSS, APIs, etc.)
12. Ajouter du **monitoring** pour visualiser l'ensemble des métriques que vous souhaitez monitorer (ex: airflow, API, performances du modèle, etc.). Utilisez la techno la plus adaptée à vos besoins : Elasticsearch + Kibana, Prometheus + Grafana, etc.
13. Vous pouvez ajouter des **tests de montée en charge** (ex: avec Locust)
14. Gérer correctement le **versionning** / **montée de version** et **rollback** des modèles
15. Ajouter de **l'évaluation** / human in the loop pour labelliser / corriger vos prédictions
16. Gestion des accès à l'API de Serving et dashboard par utilisateur de l'API
17. Alerting par mail lors d'un réentraînement ou d'une erreur
18. etc.



orchestrateur / pipelines (pour extraction de données, etc.)	Airflow
ML/Deep learning frameworks	Libre
Model registry	Libre
Feature store / BDD	Libre (optionnel)
ML Metadata Store	MLflow
Source repository	GitHub
CI/CD	GitHub Actions
Conteneurisation	Docker / Docker compose
Docker registry	Dockerhub (https://hub.docker.com/) (optionnel)
Serving (API)	Libre
Webapp d'inférence	Libre
Déploiement	Kubernetes
Monitoring	Libre (optionnel)

Contraintes et Bonnes Pratiques

- **Modularité et maintenabilité** : code bien structuré et réutilisable.
- **CI/CD** : mettre en place des tests et des pipelines de déploiement.
- **Logging et monitoring** : ajouter des logs pertinents et un système de monitoring.
- **Testing** : ajouter des tests unitaires, test d'intégration, test end-to-end, etc.
- **Sécurité et gestion des accès** : gérer correctement les permissions des accès.
- **Documentation** : rédiger un README détaillé et documenter chaque partie du projet.
- **Présentation** : rédiger une présentation qui explique votre architecture, vos choix et votre projet.

Livrables

À la fin du projet, vous devrez fournir via un mail à prillard.martin@gmail.com :

1. Un lien d'un **repository GitHub** contenant :
    - tout le code (preprocessing, entraînement, API opérationnelle, Webapp intéractive permettant de tester le modèle, etc..)
    - vos DAG de pipelines / workflows fonctionnels
    - la CI/CD avec Github Actions
    - l'ensemble de vos tests (tests unitaires, test d'intégration, test end-to-end, etc.)
    - la documentation du projet
    - un **README** expliquant les choix techniques et les résultats obtenus, avec si possible des screenshots des résultats sur votre environnement de production
2. Votre présentation en PDF

N'oubliez pas de préciser dans le mail la **liste de toutes les personnes** du groupe !

Évaluation

En fin de journée, **présentation par groupe** de **6 minutes** + **4 minutes** de questions sur votre projet.

Votre projet sera évalué selon les critères suivants :

- **Qualité du code** (respect des bonnes pratiques, modularité, documentation)
- **Bonne utilisation de l'IA Gen :** pas de code / README superflues ("less is more") et compréhension de ce que vous faites.
- **Automatisation du pipeline** (Airflow et CI/CD)
- **Suivi et gestion des modèles avec MLFlow**
- **Fonctionnalité et ergonomie de l'API (ex: Swagger) et de la WebApp**
- **Objectifs réalisés et cohérence de l'architecture globale créée**
- **Capacité à collaborer et versionner efficacement sur GitHub**